

Listening Between the Lines

Analyzing Gendered Conversational Dynamics in Indian Podcasts

Project Documentation

Team: Bhavini Sai Mallu, Sameeksha Rao

Prepared: September 2026

About This Document

This document records the analysis scripts developed for the project, in the order they were built. Each section describes the purpose of a script, the methodology it implements, and the output it produced when run on the collected podcast dataset. Sections 1-9 cover nine scripts: `src/compute_bias_features.py` (per-speaker lexicon features), `src/balanced_dataset.py` (balanced 200-episode sample across gender dyads), `src/dyad_analysis.py` (episode-level dyad dataset with dominance measures), `src/dyad_level_features.py` (dyad-level summary statistics, asymmetry features, and visualizations), `src/lexical_audit.py` (manual precision audit of the directive lexicon), `src/dialogue_act_classification.py` (model-based dialogue act labels per turn), `src/dyad_dialogue_acts.py` (episode-level dyad dataset of dialogue act rates), and `src/stat_analysis/kruskal_wallis_test_dialogue_acts.py` and `src/stat_analysis/Mann_Whitney_test_dialogue_acts.py` (formal significance testing of the dialogue-act dyad differences). Sections 10-15 document the second version of the linguistic features: `src/compute_bias_features_v2.py` (published hedge/booster and politeness resources adapted to speech), the precision audits that shaped it (`src/make_audit_sample.py`), `src/episode_dyads.py` (gender dyads over all 418 episodes), `src/dyad_analysis_v2.py` (episode-level dyad dataset), and the significance tests `src/stat_analysis/kruskal_wallis_test_v2.py`, `src/stat_analysis/Mann_Whitney_test_v2.py` and `src/stat_analysis/within_host_test_v2.py` (guest-gender effect within host).

1. `src/compute_bias_features.py`

1.1 Purpose

This script is the entry point of the linguistic bias analysis. For every processed podcast episode (WhisperX transcript + diarization + gender + host/guest labels), it computes lexicon-based conversational features for each speaker: how often they hedge, boost, are polite, or use directive/interrupting language. The goal is to build a speaker-level, episode-level dataset that can later be compared across gender and role (host vs. guest) to study gendered conversational dynamics.

1.2 Methodology

The script defines four hand-built lexicons of English words/phrases, each capturing a distinct conversational style associated with linguistic politeness/dominance research:

Category	What it captures	Example phrases
Hedge	Tentative, uncertain language	maybe, perhaps, possibly, probably, i think, i feel, i guess, kind of, sort of, a bit, a little, it seems, it looks like, in a way
Booster	Confident, assertive language	definitely, clearly, obviously, absolutely, certainly, for sure,

		no doubt, everyone knows
Polite	Politeness markers	please, sorry, apologies, thank you, thanks, appreciate, excuse me, pardon me, if you don't mind
Directive	Interrupting / floor-holding language	hold on, hang on, wait, listen, let me explain, let me finish, let me tell you, here's the thing, actually, look,

Processing steps implemented in the script:

- Compile each lexicon into regex patterns, matching longer phrases first so that e.g. "it looks like" is not double-counted inside a shorter overlapping match, with word-boundary (\b) matching and case-insensitivity.
- Load each episode's WhisperX JSON output and group transcript segments by (speaker_id, role, gender), concatenating all of a speaker's spoken text in that episode into a single string.
- For each speaker's text: count total words, then count occurrences of each lexicon category, and normalize to a rate per 1,000 words (per_1k) so speakers with very different amounts of speaking time remain comparable.
- Assemble one row per (episode, speaker) with word_count and the four categories' raw counts and per-1k rates.
- Across all matched episode JSON files (via a glob pattern), concatenate all rows into a single DataFrame, drop any speaker with fewer than 50 words (too little text to estimate a rate reliably), and write the result to a CSV.

1.3 How It Was Run

The script is run from the command line against the folder of processed episode JSONs (the output of the WhisperX → gender detection → host/guest labeling stages of the pipeline):

```
python src/compute_bias_features.py \
  --input_glob "data/outputs/whisperx/*_whisperx_diarized.gender.host_guest.json" \
  --out_csv results/features/speaker_features.csv
```

1.4 Output

Output file: results/features/speaker_features.csv — 720 rows × 13 columns, covering 360 unique episodes (2 speaker rows per episode: 1 host + 1 guest).

Columns produced:

Column	Description
episode_id	Episode identifier parsed from the JSON filename

speaker_id	Diarization speaker label (e.g. SPEAKER_00)
role	HOST or GUEST
gender	male, female, or unknown
word_count	Total words spoken by this speaker in the episode
hedge_count / hedge_per_1k	Raw count and per-1,000-word rate of hedging language
booster_count / booster_per_1k	Raw count and per-1,000-word rate of boosting language
polite_count / polite_per_1k	Raw count and per-1,000-word rate of politeness markers
directive_count / directive_per_1k	Raw count and per-1,000-word rate of directive/interrupting language

1.5 Dataset Composition of the Output

Metric	Value
Total speaker-episode rows	720
Unique episodes	360
HOST rows	360
GUEST rows	360
Male speakers	491
Female speakers	227
Unknown gender speakers	2

1.6 Descriptive Statistics

Summary statistics across all 720 speaker-episode rows for word count and each lexicon category's per-1,000-word rate:

Statistic	Word Count	Hedge /1k	Booster /1k	Polite /1k	Directive /1k
mean	6238.3	6.19	1.01	1.53	1.24
std	4576.0	4.35	0.96	1.62	1.06
min	283.0	0.00	0.00	0.00	0.00
25%	2534.2	2.98	0.40	0.46	0.52
50%	4977.5	5.12	0.77	1.07	0.92
75%	9041.8	8.32	1.30	2.08	1.75
max	29936.0	34.02	7.76	13.61	6.84

1.7 Sample Output Rows

Episode	Speaker	Role	Gender	Words	Hedge/1k	Booster /1k	Polite/1k	Directive/1k
ep_108	SPEAKE	HOST	male	2901	8.27	1.03	1.72	2.07

	R_00							
ep_108	SPEAKE R_01	GUEST	male	17203	2.62	0.64	0.81	0.87
ep_248	SPEAKE R_01	HOST	male	2737	6.94	0.37	0.73	1.10
ep_248	SPEAKE R_00	GUEST	male	7903	13.16	2.40	0.13	0.38
ep_001	SPEAKE R_01	HOST	male	1941	6.18	2.58	5.67	0.52

1.8 Notes & Limitations

- The lexicons are small and hand-curated (a starting point, as noted in the script's own comment: "start small, expand later"); they do not capture every hedge/booster/polite/directive phrase, so counts should be read as an approximation of style, not exhaustive.
- Phrase matching is regex/substring-based (with word boundaries), not part-of-speech or context aware — e.g. "actually" is always counted as directive regardless of sentence role.
- Speakers with fewer than 50 words in an episode are dropped, since per-1k rates are unstable on very little text.
- This script's compiled lexicon (COMPILED) is directly reused by `src/lexical_audit.py`, and its `episode_id` convention is reused by `src/dialogue_act_classification.py`, making it the foundation for later analyses in the pipeline.

2. src/balanced_dataset.py

2.1 Purpose

The raw `speaker_features.csv` dataset is skewed across host/guest gender combinations (e.g. far more male-hosted episodes than female-hosted ones), which would bias any downstream comparison between gender pairings. This script builds a balanced subset of episodes — equal numbers of episodes for each host-gender / guest-gender combination — so that later dyad-level comparisons are not confounded by an uneven sample.

2.2 Methodology

- Load `results/features/speaker_features.csv` and keep only HOST and GUEST rows.
- For each episode, determine the host's gender and the guest's gender (taking the gender of whichever HOST/GUEST row has the higher `word_count`, in case of duplicate role rows), and drop any episode missing either.
- Build a per-episode "dyad" label by combining host and guest gender, e.g. MALE->FEMALE means a male host with a female guest.
- Restrict to the four valid same/cross-gender dyads: MALE->MALE, MALE->FEMALE, FEMALE->MALE, FEMALE->FEMALE (dropping any 'unknown' gender pairings).
- Randomly sample 50 episodes per dyad (fixed `random_state=42` for reproducibility); warn if a dyad has fewer than 50 available episodes.
- Save the 200 selected episode IDs, with their host/guest gender and dyad label, to `results/features/balanced_200_episodes.csv`.

2.3 How It Was Run

```
python src/balanced_dataset.py
```

The script takes no command-line arguments — input and output paths are hard-coded.

2.4 Output

Output file: `results/features/balanced_200_episodes.csv` — 200 episodes × 4 columns (`episode_id`, `host_gender`, `guest_gender`, `dyad`). This is the balanced, 50-episodes-per-dyad sample used as the basis for the dyad-level analysis in Section 3.

Dyad (host->guest)	Episodes sampled
MALE->MALE	50
MALE->FEMALE	50
FEMALE->MALE	50
FEMALE->FEMALE	50

2.5 Notes & Limitations

- Sampling is random (not stratified by any other variable, e.g. episode length or podcast series), so the 200-episode subset may still vary in other respects across dyads.

- The random_state=42 seed makes the sample reproducible across runs, but re-running the script after speaker_features.csv changes (e.g. more episodes processed) will draw a different set of 50-per-dyad episodes.
- This balanced set of episode IDs is the required input to src/dyad_analysis.py.

3. src/dyad_analysis.py

3.1 Purpose

This script turns the balanced, speaker-level dataset into an episode-level ("dyad-level") dataset: one row per episode with the host's and guest's features side by side, plus derived measures of conversational dominance. This is the shape needed to directly compare, for example, how much a male host dominates a female guest versus how much a female host dominates a male guest.

3.2 Methodology

- Load results/features/speaker_features.csv and results/features/balanced_200_episodes.csv, and restrict the speaker-level data to only the 200 balanced episodes.
- Sanity check: confirm every episode has exactly one HOST row and one GUEST row; print a warning listing any episode that doesn't.
- Pivot from long format (one row per speaker) to wide format (one row per episode) by splitting into a host table and a guest table and merging them on episode_id, prefixing every feature column with host_ or guest_.
- Merge in the dyad / gender-pair labels from balanced_200_episodes.csv.
- Cross-check: verify the gender recorded in speaker_features.csv agrees with the gender recorded in balanced_200_episodes.csv for both host and guest; warn on any mismatch, then drop the duplicate gender columns.
- Compute derived conversational-dynamics features: total_words (host + guest), host_speaking_share and guest_speaking_share (each speaker's fraction of total words), and dominance_ratio = host_word_count / guest_word_count (>1 means the host talks more than the guest).
- Reorder columns into a clean, readable layout and save to results/dyads/dyad_analysis.csv.

3.3 How It Was Run

```
python src/dyad_analysis.py
```

Like balanced_dataset.py, this script has no command-line arguments; it depends on both speaker_features.csv and balanced_200_episodes.csv already existing.

3.4 Output

Output file: results/dyads/dyad_analysis.csv — 200 rows (one per balanced episode) × 26 columns. No HOST/GUEST role mismatches or gender-label mismatches were found when this was last run.

Key columns produced:

Column group	Description
--------------	-------------

episode_id, dyad, host_gender, guest_gender	Episode and dyad identity
host_word_count, guest_word_count, total_words	Raw speaking volume per role
host_speaking_share, guest_speaking_share	Each role's fraction of total words spoken
dominance_ratio	$\text{host_word_count} / \text{guest_word_count}$ — >1 means the host talks more
host_/guest_{hedge,booster,polite,directive}_count / _per_1k	The four lexicon-based style features from compute_bias_features.py, split by role

3.5 Dominance & Speaking Share by Dyad

Average host speaking share and dominance ratio for each of the four balanced dyads (50 episodes each). A dominance_ratio below 1 means the host, on average, speaks less than the guest in that dyad type:

Dyad	Host speaking share	Dominance ratio	Host hedge/1k	Guest hedge/1k	Host directive/1k	Guest directive/1k
MALE->MALE	0.262	0.719	5.40	5.59	0.98	0.96
MALE->FEMALE	0.288	0.427	6.23	6.70	1.12	1.17
FEMALE->MALE	0.186	0.236	5.70	8.87	1.81	1.56
FEMALE->FEMALE	0.214	0.332	6.81	9.33	1.47	1.75

3.6 Sample Output Rows

Episode	Dyad	Host words	Guest words	Host share	Dominance ratio
ep_248	MALE->MALE	2737	7903	0.26	0.35
ep_049	MALE->MALE	912	10913	0.08	0.08
ep_270	MALE->FEMALE	3449	9434	0.27	0.37

3.7 Notes & Limitations

- Built-in sanity checks (role-count check and gender cross-check) print warnings but do not halt execution — any flagged episodes should be reviewed manually before trusting downstream results; none were flagged on the current 200-episode run.

- dominance_ratio is undefined (division by zero) if a guest has zero recorded words; this did not occur in the current dataset since compute_bias_features.py already drops speakers with fewer than 50 words.
- The dataset is limited to the 200 balanced episodes from Section 2 — it intentionally excludes the remaining episodes to keep dyad comparisons unconfounded by sample-size imbalance.
- This dyad-level file is the basis for later dyad-level statistical analysis (e.g. results/dyads/dyad_analysis_with_asymmetry.csv, dyad_summary_stats.csv, and dyad_median_mad_stats.csv).

4. src/dyad_level_features.py

4.1 Purpose

This script takes the episode-level dyad dataset from Section 3 (dyad_analysis.csv) and turns it into the summary statistics and visualizations used to compare conversational dominance and style across the four host/guest gender dyads. It also adds host-vs-guest "asymmetry" features (how much more, or less, the host hedges/boosts/is polite/is directive than the guest in the same episode), which are more directly interpretable than looking at host and guest rates separately. This script originally produced its summary tables and boxplots only; the median +/- MAD bar-chart step (4.2, step 6, and Section 4.6) was originally a separate script, src/plot_mean_median.py, and has since been merged into this script so that a single run produces every dyad-level summary output.

4.2 Methodology

- Load results/dyads/dyad_analysis.csv (the 200-episode dyad-level dataset).
- Balance check: print the number of episodes per dyad and warn if the four dyad groups are not equal in size.
- Add asymmetry features: for hedge, booster, polite, and directive, compute $\text{host_rate} - \text{guest_rate}$ (per-1k). A positive value means the host uses that style more than the guest in that episode; negative means the guest uses it more.
- Save the full episode-level table (original columns + the four new asymmetry columns) to results/dyads/dyad_analysis_with_asymmetry.csv.
- Summary stats: for every metric (host/guest speaking share, dominance ratio, all host/guest per-1k rates, and the four asymmetry columns), compute mean, median, std, and count per dyad, and save to results/dyads/dyad_summary_stats.csv.
- Robust median + MAD stats: repeat the same per-dyad aggregation using median and median absolute deviation (MAD) instead of mean/std, since MAD is far less sensitive to outlier episodes (e.g. one very lopsided conversation). Saved to results/dyads/dyad_median_mad_stats.csv.
- Boxplots: for six key metrics (dominance ratio, the four asymmetry columns, and host speaking share), draw a grid of boxplots grouped by dyad (abbreviated MM/MF/FM/FF) as a visual sanity check before formal statistical testing. Saved to results/figures/dyad_boxplots.png.
- Median +/- MAD bar chart: for five key metrics (dominance ratio and the four asymmetry columns), draw one bar per dyad using the median as the bar height and the MAD as the error bar, reusing the median/MAD summary computed above rather than re-reading it from disk. Saved to results/figures/dyad_median_mad_barplot.png.

4.3 How It Was Run

```
python src/dyad_level_features.py \  
--input results/dyads/dyad_analysis.csv \  

```

```
--outdir results/dyads/ \
--figdir results/figures/
```

All three arguments have defaults matching the paths above, so the script can also be run with no arguments once dyad_analysis.csv exists.

4.4 Output Files

File	Shape	Description
dyad_analysis_with_asymmetry.csv	200 rows x 30 cols	Episode-level data + hedge/booster/polite/directive asymmetry columns
dyad_summary_stats.csv	4 dyads x 60 cols	Mean / median / std / count per dyad, for every metric
dyad_median_mad_stats.csv	4 dyads x 45 cols	Robust median / MAD / count per dyad, for every metric
dyad_boxplots.png	6-panel figure	Boxplots of dominance ratio, the 4 asymmetry metrics, and host speaking share, by dyad
dyad_median_mad_barplot.png	5-panel figure	Median +/- MAD bar chart of dominance ratio and the 4 asymmetry metrics, by dyad

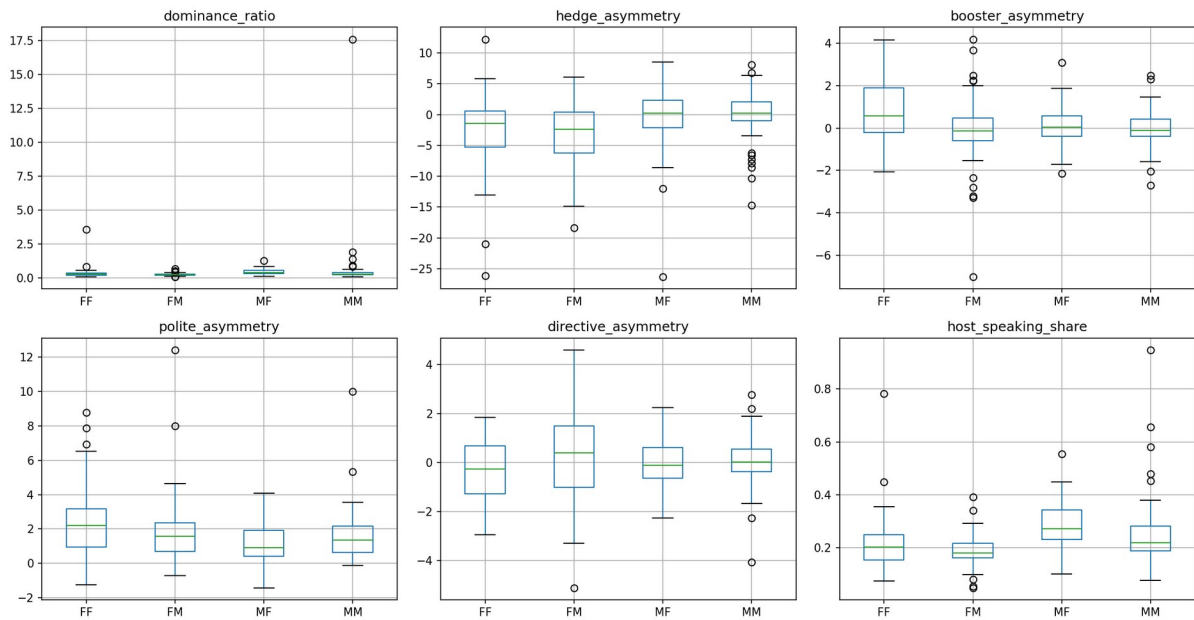
4.5 Median ± MAD by Dyad (Key Metrics)

Robust summary (median and median absolute deviation) for dominance ratio and the four host-vs-guest asymmetry metrics. A positive asymmetry median means hosts in that dyad tend to use that style more than their guests; a negative one means guests use it more.

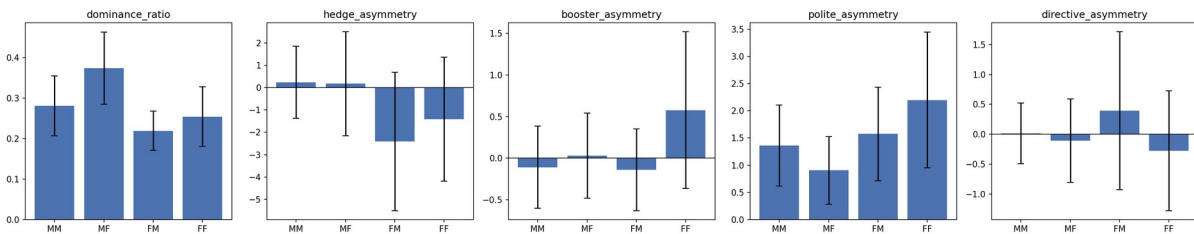
Dyad	Dominance ratio	Hedge asym.	Booster asym.	Polite asym.	Directive asym.
MALE->MALE	0.28 ± 0.07	0.24 ± 1.61	-0.11 ± 0.49	1.36 ± 0.75	0.02 ± 0.51
MALE->FEMALE	0.37 ± 0.09	0.18 ± 2.33	0.03 ± 0.51	0.91 ± 0.62	-0.11 ± 0.70
FEMALE->MALE	0.22 ± 0.05	-2.41 ± 3.10	-0.14 ± 0.49	1.58 ± 0.86	0.39 ± 1.32
FEMALE->FEMALE	0.25 ± 0.07	-1.41 ± 2.77	0.58 ± 0.94	2.20 ± 1.25	-0.28 ± 1.00

4.6 Figures

Boxplots of key dyad-level metrics (dyad_boxplots.png):



Median $\pm$ MAD bar chart of key dyad-level metrics (dyad_median_mad_barplot.png):



4.7 Interpretation: What These Results Suggest

Reading the median dominance ratio, speaking share, and asymmetry values together (Section 4.5) points to a few patterns in this 200-episode balanced sample. These are descriptive observations from medians/MAD, not the result of formal significance testing (that is left to later analysis scripts) — they describe what this sample looks like, not a confirmed effect.

- **Guests talk more than hosts, in every dyad.** The median dominance ratio is below 1 for all four dyads (0.22-0.37), and the median host speaking share is only 18-27% of total words versus 73-82% for guests. This holds regardless of host or guest gender, and is consistent with the interview format of these podcasts: hosts ask questions and guests give longer answers.
- **Male hosts hold the floor more with female guests than female hosts do with male guests.** MALE->FEMALE has the highest host speaking share (0.272) and highest dominance ratio (0.374) of the four dyads, while FEMALE->MALE has the lowest (0.180 share, 0.219 ratio). In

other words, the biggest cross-gender asymmetry in who dominates floor time runs in one direction: male hosts speak relatively more when their guest is a woman than female hosts do when their guest is a man.

- **Female hosts hedge far less than their guests; male hosts are roughly even with theirs.** Hedge asymmetry (host rate minus guest rate) is close to zero for male hosts (+0.24 with male guests, +0.18 with female guests), but strongly negative for female hosts (-2.41 with male guests, -1.41 with female guests) — meaning female hosts hedge noticeably less than the person they're interviewing, running counter to the common stereotype that women hedge more in conversation.
- **Hosts are consistently more polite than their guests, in every dyad.** Polite asymmetry is positive for all four dyads (0.91-2.20), and largest for FEMALE->FEMALE (2.20). Since this direction holds regardless of gender, it looks more like a host-role effect (thanking, welcoming, apologizing as the moderator) than a gender effect — though it is most pronounced between two women.
- **Boosting and directive/interrupting language show no consistent gender pattern.** Booster asymmetry is small in three of four dyads and only stands out for FEMALE->FEMALE (+0.58, female hosts boosting more than female guests). Directive asymmetry stays small ($|\text{value}| < 0.4$) across all four dyads, suggesting interruption-style language in this sample is not strongly linked to the host/guest gender pairing.

4.8 Notes & Limitations

- Mean/std (dyad_summary_stats.csv) and median/MAD (dyad_median_mad_stats.csv) are both saved because dominance_ratio and the asymmetry metrics can be skewed by a small number of very lopsided episodes; the median/MAD view is the more robust one to read for group comparisons.
- The boxplots and bar chart are both intended as visual sanity checks ahead of formal statistical testing (e.g. group comparisons across dyads) rather than as the final analysis themselves.
- The balance check only warns (via printed output) if dyad group sizes differ — it does not stop the script — but on the current balanced_200_episodes.csv input, all four dyads are confirmed equal at n=50.
- The median +/- MAD bar-chart logic previously lived in the standalone script src/plot_mean_median.py, which re-read dyad_median_mad_stats.csv from disk; it has been merged into this script (plot_median_madBars()) so it reuses the summary already held in memory, and the standalone script has been removed.
- The interpretation in Section 4.7 is descriptive only (medians and MAD on a 200-episode balanced sample) — no hypothesis tests have been run yet to confirm which of these patterns are statistically robust versus noise.

5. src/lexical_audit.py

5.1 Purpose

The lexicon-based features in `compute_bias_features.py` (Section 1) count phrases like "actually" or "listen" as directive/interrupting language purely by regex matching, with no understanding of how the phrase is actually being used in context. This script is a manual precision audit: it pulls random real transcript examples of a lexicon category's matches, with surrounding context, so a human can label each one as a true or false positive and estimate how reliable that lexicon category actually is.

5.2 Methodology

- Import the already-compiled lexicon patterns (COMPILED) directly from `compute_bias_features.py`, so the audit checks the exact same regex patterns used to generate the feature dataset.
- For a chosen lexicon category (here, "directive"), scan every WhisperX transcript segment across all episode JSON files and find every regex match for that category's phrases.
- For each match, capture an 80-character window of text on either side, so the phrase can be judged in its real conversational context (e.g. whether "listen" was used as an attention-grabbing directive, or just literally means to listen to something).
- Shuffle all matches across the whole dataset and take a random sample (50 examples), so the audit isn't biased toward any one episode or phrase.
- Add an empty `is_true_positive` column for manual labeling, and save the sample to CSV for a human reviewer to fill in by hand.

5.3 How It Was Run

`python src/lexical_audit.py`

The script has no command-line arguments; the category audited ("directive"), sample size (50), input glob, and output path are all set directly in the script. After it ran, the `is_true_positive` column in the output CSV was filled in by hand, one row at a time, to judge whether each matched phrase was really being used as directive/interrupting language.

5.4 Output

Output file: `results/dialogue_acts/directive_audit_sample.csv` — 50 randomly sampled "directive" phrase matches, each with its source episode, the matched phrase, 80 characters of surrounding context, and a manually filled-in `is_true_positive` label (True/False).

Columns produced:

Column	Description
episode	Path to the source episode JSON the match came from
phrase	The specific directive-lexicon phrase that

	matched (e.g. "actually", "listen")
context	±80 characters of transcript text around the match, for manual judgment
is_true_positive	Manually filled in: True if the phrase is genuinely directive/interrupting language in context, False if it's a false match

5.5 Manual Audit Results

Of the 50 randomly sampled "directive" matches, only 12 were manually judged to be true directive/interrupting usage — an overall precision of 24%. Precision varied sharply by phrase:

Phrase	Sampled	True positives	Precision
actually	29	8	28%
listen	14	2	14%
wait	4	0	0%
let me tell you	2	2	100%
hold on	1	0	0%

"actually" and "listen" made up 43 of the 50 samples (86%) but had the lowest precision (28% and 14% respectively) — most of their matches were ordinary uses ("he actually had...", "if you listen to it...") rather than someone taking or holding the conversational floor. "let me tell you" was the only phrase that matched reliably (2/2), though it appeared far less often in the transcripts.

5.6 Notes & Limitations

- A 24% precision on the directive category means the `directive_count / directive_per_1k` features in `speaker_features.csv` (Section 1) are dominated by false positives — mostly generic uses of "actually" and "listen" — and should not be trusted as a clean measure of interrupting/floor-holding behavior without further filtering.
- The audit sampled only the "directive" category with $n=50$; the other three lexicon categories (hedge, booster, polite) have not been manually audited this way, so their precision is unknown.
- Labeling was done manually by one reviewer on a single random sample, so the precision estimates above carry sampling noise and some subjectivity in what counts as "true" directive usage.
- This audit is the direct motivation for later work distinguishing lexicon matches from true dialogue acts (`src/dialogue_act_classification.py` and `src/validate_dialogue_act_vs_lexicon.py`), which classify directive/interruption behavior using more than simple phrase matching.

6. src/dialogue_act_classification.py

6.1 Purpose

Section 5's audit showed that lexicon phrase-matching is an unreliable way to detect conversational behavior like directive/interrupting speech (24% precision). This script replaces keyword matching with a pretrained dialogue-act classifier that reads each turn in full sentence context (the previous turn plus the current one), producing a much more meaningful label for what each turn is actually doing in the conversation — asking, answering, acknowledging, exclaiming, etc.

6.2 Methodology

- Use the pretrained diwank/silicone-deberta-pair model (a DeBERTa model fine-tuned on the SILICONE dialogue-act dataset) via the simpletransformers library, classifying each turn into one of 11 dialogue act labels: acknowledge, answer, backchannel, reply_yes, exclaim, say, reply_no, hold, ask, intent, ask_yes_no.
- Restrict processing to the 200 episodes already selected in balanced_200_episodes.csv (Section 2), so the dialogue-act labels line up with the same balanced sample used elsewhere.
- For each episode, load every transcript segment in order and build (previous_text, current_text) pairs — the model needs the prior turn as context to disambiguate a turn (e.g. "yes" as reply_yes only makes sense next to the question it answers).
- Run the model in batches (default batch size 32) over every turn in each episode, assigning one of the 11 labels to each.
- Save every turn's label to a per-turn CSV, then aggregate two ways: (1) the normalized rate of each dialogue act per (episode, role) — i.e. what fraction of a host's or guest's turns in an episode were each act — and (2) an overall rate by gender as a quick sanity check.

6.3 How It Was Run

```
python src/dialogue_act_classification.py \  
  --input_glob "data/outputs/whisperx/*_whisperx_diarized.gender.host_guest.json" \  
  --balanced_csv results/features/balanced_200_episodes.csv \  
  --out_dir results/dialogue_acts/ \  
  --max_episodes 200
```

The model requires simpletransformers (pip install simpletransformers). FP16 is disabled in the model args, since DeBERTa's disentangled-attention masking overflows half precision.

6.4 Output

File	Shape	Description
dialogue_act_labels.csv	234,459 rows x 6 cols	One row per transcript turn: episode_id, speaker, role, gender, text, and the predicted dialogue_act label

dialogue_act_rates_by_episode_role.csv	400 rows x 13 cols	One row per (episode, role): the fraction of that speaker's turns falling into each of the 11 dialogue act categories
---	--------------------	---

6.5 Label Distribution

Across 200 episodes and 234,459 classified turns, the overall distribution of dialogue acts is:

Dialogue act	Turns	Share of all turns
say	97,428	41.6%
answer	91,073	38.8%
ask	24,202	10.3%
acknowledge	7,228	3.1%
intent	6,468	2.8%
reply_yes	4,633	2.0%
backchannel	2,203	0.9%
ask_yes_no	515	0.2%
exclaim	376	0.2%
reply_no	281	0.1%
hold	52	0.0%

6.6 Host vs. Guest Behavior

Averaging each episode's per-role rates gives a clear, face-valid host/guest split — unlike the noisy lexicon-based directive counts, this lines up with what the host/guest roles actually mean in an interview-format podcast:

Dialogue act	Host (avg. share of turns)	Guest (avg. share of turns)
ask	22.2%	6.5%
answer	35.1%	39.1%
say	29.3%	46.6%
acknowledge	5.0%	2.4%
backchannel	1.3%	0.8%
intent	3.6%	2.4%

Hosts ask far more than guests (22.2% of host turns vs. 6.5% of guest turns) and acknowledge more (5.0% vs. 2.4%), while guests lean more heavily on "say" — extended narrative/statement turns (46.6% vs. 29.3% for hosts). This is the expected shape of an interview: the host prompts and the guest narrates, and the model recovers that pattern cleanly.

6.7 Notes & Limitations

- This script depends on a large pretrained model and simpletransformers; it is far more compute-intensive than the regex-based lexicon features and was only run on the 200 balanced episodes rather than the full dataset.
- The 11 SILICONE labels are broad conversational-move categories (ask, answer, say, etc.), not the same as the 4 stylistic lexicon categories (hedge/booster/polite/directive) — this script complements rather than replaces Section 1's features.
- "hold" is extremely rare in this dataset (52 of 234,459 turns, ~0.02%), so any per-episode or per-dyad rate for it individually should be treated as noisy.
- The output is the direct input to src/dyad_dialogue_acts.py (Section 7), which reshapes it into an episode-level, host-vs-guest comparison table.

7. src/dyad_dialogue_acts.py

7.1 Purpose

This script mirrors what src/dyad_analysis.py (Section 3) does for the lexicon features, but for the model-based dialogue act rates from Section 6: it reshapes the per-(episode, role) dialogue act rate table into one row per episode, with host and guest rates side by side and the gender-dyad label attached, so dialogue-act behavior can be compared directly across the four host/guest gender pairings.

7.2 Methodology

- Load dialogue_act_rates_by_episode_role.csv (Section 6's per-episode/role output) and balanced_200_episodes.csv (Section 2), and restrict to the 200 balanced episodes.
- Sanity check: confirm every episode has exactly one HOST row and one GUEST row; print a warning listing any episode that doesn't.
- Pivot from long format (one row per role) to wide format (one row per episode) by splitting into host and guest tables and merging them on episode_id, prefixing every dialogue-act rate column with host_ or guest_.
- Merge in the dyad / host-guest gender labels from balanced_200_episodes.csv.
- Reorder columns (episode identity + dyad info first, then host_/guest_ pairs for each of the 11 dialogue acts) and save to results/dyads/dyad_dialogue_acts.csv.

7.3 How It Was Run

```
python src/dyad_dialogue_acts.py
```

The script has no command-line arguments; input and output paths are hard-coded, and it depends on dialogue_act_rates_by_episode_role.csv and balanced_200_episodes.csv already existing.

7.4 Output

Output file: results/dyads/dyad_dialogue_acts.csv — 200 rows (one per balanced episode) x 26 columns: episode_id, dyad, host_gender, guest_gender, and host_/guest_ prefixed rate columns for each of the 11 dialogue acts.

7.5 Dialogue Act Rates by Dyad

Average host and guest rates for a few key dialogue acts, broken out by the four balanced gender dyads (50 episodes each):

Dyad	Host ask	Guest answer	Host say	Guest say	Host ack.	Guest ack.
MALE->MALE	19.3%	44.9%	28.8%	38.9%	4.5%	2.5%
MALE-	21.1%	42.8%	29.9%	41.1%	4.7%	2.5%

>FEMALE						
FEMALE->MALE	25.0%	38.0%	28.3%	50.4%	5.7%	1.9%
FEMALE->FEMALE	23.5%	30.9%	30.1%	56.1%	5.3%	2.6%

Female hosts ask more than male hosts regardless of guest gender (25.0% for FEMALE->MALE and 23.5% for FEMALE->FEMALE, versus 19.3% for MALE->MALE and 21.1% for MALE->FEMALE), and also acknowledge guests more (5.3-5.7% vs. 4.5-4.7%). Guests lean most heavily on narrative "say" turns in FEMALE->FEMALE episodes (56.1%) and least in MALE->MALE episodes (38.9%). Read alongside Section 4.7 — where male hosts were found to hold more floor time (speaking share) than female hosts overall — this suggests female hosts ask proportionally more questions and speak less per turn, while male hosts speak fewer but longer turns.

7.6 Notes & Limitations

- Like `dyad_analysis.py`, this script only warns (rather than halting) if an episode is missing a HOST or GUEST row; no such episodes were flagged in the current run.
- Some dialogue acts are rare overall (e.g. hold, reply_no, exclaim — see Section 6.5), so per-dyad averages for those specific columns are based on very few observed turns per episode and should be read cautiously.
- This dataset is dialogue-act-only; it does not include the lexicon-based hedge/booster/polite/directive features from `dyad_analysis.csv` (Section 3) — combining the two would require an additional merge on `episode_id`.
- As with Section 4, these are descriptive averages, not the result of formal significance testing.

8. src/stat_analysis/kruskal_wallis_test_dialogue_acts.py

8.1 Purpose

Section 7's dyad-level dialogue-act comparisons (e.g. "female hosts ask more") were descriptive only, based on averages. This script runs a formal, non-parametric omnibus test — the Kruskal-Wallis H test — to check whether host and guest dialogue-act behavior actually differs significantly across the four gender dyads, or whether the differences seen in Section 7 could just be sampling noise. It mirrors the existing lexicon-based dyad test (src/stat_analysis/kruskal_wallis_test_lexicon.py), applied instead to the model-based dialogue-act rates from Section 7.

8.2 Methodology

- Load results/dyads/dyad_dialogue_acts.csv (Section 7's episode-level, host/guest-by-dyad dataset).
- Restrict to the 6 higher-frequency dialogue acts: ask, answer, say, acknowledge, intent, reply_yes. The remaining 5 acts (hold, reply_no, exclaim, ask_yes_no, backchannel) are all under 1% of turns overall (Section 6.5) and too rare per episode to test reliably.
- For each of the 6 acts, compute an asymmetry column = host_rate - guest_rate, matching the same host-minus-guest convention used for the lexicon asymmetry features (Section 4).
- Test 7 metrics in total: host_ask (an overall host-behavior level metric, mirroring host_speaking_share in the lexicon test) plus the 6 asymmetry columns.
- For each metric, run scipy's kruskal() across the four dyad groups (Kruskal-Wallis is the non-parametric, rank-based equivalent of a one-way ANOVA, and does not assume normally distributed data) and record the H statistic and p-value.
- Save all 7 results to results/stat_analysis/kruskal_wallis_dialogue_acts_results.csv.

8.3 How It Was Run

```
python src/stat_analysis/kruskal_wallis_test_dialogue_acts.py
```

No command-line arguments; input and output paths are hard-coded.

8.4 Results

Kruskal-Wallis omnibus test across the 4 gender dyads, for each metric (H statistic and p-value; p < 0.05 means the metric differs significantly across at least one pair of dyads):

Metric	H statistic	p-value	Significant (p<0.05)
host_ask	15.907	0.0012	Yes
ask_asymmetry	29.654	0.0000	Yes
answer_asymmetry	27.743	0.0000	Yes
say_asymmetry	60.657	0.0000	Yes
acknowledge_asymmetry	4.556	0.2073	No
intent_asymmetry	1.917	0.5898	No

reply_yes_asymmetry	7.348	0.0616	No
---------------------	-------	--------	----

4 of the 7 metrics are significant: host_ask, ask_asymmetry, answer_asymmetry, and say_asymmetry. acknowledge_asymmetry, intent_asymmetry, and reply_yes_asymmetry show no significant difference across dyads and are excluded from the pairwise follow-up test in Section 9.

8.5 Notes & Limitations

- Kruskal-Wallis only tells you that at least one dyad differs from another for a given metric — it does not say which pair(s) differ; that requires the pairwise follow-up test in Section 9.
- No multiple-comparison correction is applied across the 7 metrics tested here (each is an independent omnibus test); correction is applied downstream, within each metric's 6 pairwise comparisons, in Section 9.
- As in Section 7, rare dialogue acts are excluded up front rather than tested and found non-significant, since their per-episode rates are based on very few turns.

9. src/stat_analysis/Mann_Whitney_test_dialogue_acts.py

9.1 Purpose

This script follows up on Section 8's significant metrics with pairwise comparisons: for each of the 4 significant metrics, it tests every pair of the 4 gender dyads against each other (6 pairs) using the Mann-Whitney U test, corrects for running 6 tests per metric, and reports an effect size — pinning down exactly which dyad differences are driving each significant result from Section 8.

9.2 Methodology

- Load results/dyads/dyad_dialogue_acts.csv and recompute the same 6 asymmetry columns as Section 8.
- Test only the 4 metrics that were significant in the Kruskal-Wallis omnibus test: host_ask, ask_asymmetry, answer_asymmetry, say_asymmetry.
- For each metric, run a two-sided Mann-Whitney U test (the non-parametric equivalent of an independent-samples t-test) on every one of the 6 possible pairs among the 4 dyads.
- For each pair, also compute the rank-biserial correlation as an effect size (ranges -1 to 1; magnitude indicates how separated the two groups' distributions are).
- Correct the 6 p-values within each metric for multiple comparisons using the Benjamini-Hochberg false discovery rate (FDR) procedure, and flag a pair as significant if its corrected p-value is below 0.05.
- Save all 24 (4 metrics x 6 pairs) results to results/stat_analysis/pairwise_dyad_tests_dialogue_acts.csv.

9.3 How It Was Run

python src/stat_analysis/Mann_Whitney_test_dialogue_acts.py

No command-line arguments; input and output paths are hard-coded.

9.4 Results: Significant Pairwise Differences

Of the 24 pairwise tests run (4 metrics x 6 dyad pairs), 14 remained significant after FDR correction:

Metric	Dyad A	Dyad B	Median A	Median B	Corrected p	Effect size
answer_asymmetry	FEMALE->FEMALE	MALE->FEMALE	0.004	-0.070	0.0000	-0.527
answer_asymmetry	FEMALE->FEMALE	MALE->MALE	0.004	-0.065	0.0000	-0.536
answer_asymmetry	FEMALE->FEMALE	FEMALE->MALE	0.004	-0.043	0.0017	-0.387
ask_asymmetry	FEMALE->MALE	MALE->MALE	0.186	0.107	0.0000	-0.542
ask_asymmetry	FEMALE->MALE	MALE->FEMALE	0.184	0.107	0.0001	-0.474

metry	>FEMALE	>MALE				
ask_asymmetry	FEMALE->MALE	MALE->FEMALE	0.186	0.135	0.0016	-0.390
ask_asymmetry	FEMALE->FEMALE	MALE->FEMALE	0.184	0.135	0.0120	-0.308
host_ask	FEMALE->MALE	MALE->MALE	0.237	0.176	0.0026	-0.409
host_ask	FEMALE->FEMALE	MALE->MALE	0.234	0.176	0.0085	-0.347
host_ask	FEMALE->MALE	MALE->FEMALE	0.237	0.207	0.0446	-0.266
say_asymmetry	FEMALE->FEMALE	MALE->MALE	-0.252	-0.086	0.0000	0.722
say_asymmetry	FEMALE->FEMALE	MALE->FEMALE	-0.252	-0.096	0.0000	0.686
say_asymmetry	FEMALE->MALE	MALE->MALE	-0.248	-0.086	0.0000	0.552
say_asymmetry	FEMALE->MALE	MALE->FEMALE	-0.248	-0.096	0.0000	0.542

9.5 Interpretation: What These Results Suggest

- **Asking more questions is a host-gender effect, not a guest-gender effect.** Both host_ask and ask_asymmetry are significantly higher when the host is female, specifically FEMALE->MALE and FEMALE->FEMALE both significantly exceed MALE->MALE (and FEMALE->MALE also exceeds MALE->FEMALE). But the two female-host dyads (FEMALE->MALE vs FEMALE->FEMALE) are never significantly different from each other, and neither are the two male-host dyads (MALE->FEMALE vs MALE->MALE) — the guest's gender doesn't move the needle once the host's gender is fixed.
- **The answer_asymmetry effect is specific to two women talking, not a general host or guest gender pattern.** Every significant answer_asymmetry pair involves FEMALE->FEMALE (vs. all three other dyads), with FEMALE->FEMALE hosts answering about as much as their guests (median +0.004) while every other dyad's host answers noticeably less than the guest (medians -0.04 to -0.07). None of the other three dyads differ significantly from each other. This looks like a distinct two-women-talking dynamic rather than a straightforward host-gender or guest-gender main effect.
- **Guests over-narrate relative to their host far more when the host is a woman, regardless of the guest's gender.** say_asymmetry shows the strongest effect sizes in this analysis (rank-biserial 0.54-0.72): both female-host dyads (FEMALE->MALE, FEMALE->FEMALE, medians around -0.25) differ significantly from both male-host dyads (MALE->FEMALE, MALE->MALE, medians around -0.09), while the two female-host dyads don't differ from each other and the two male-host dyads don't differ from each other. In other words, guests consistently

take on much more of the narrative "say" role, relative to their host, specifically when that host is a woman.

9.6 Notes & Limitations

- As with the lexicon-based pairwise test (`src/stat_analysis/Mann_Whitney_test_lexicon.py`), FDR correction is applied within each metric's 6 comparisons separately, not across all 24 tests jointly — a stricter global correction would likely push a few of the borderline results (e.g. `host_ask FEMALE->MALE` vs `MALE->FEMALE`, corrected $p=0.045$) out of significance.
- These are still descriptive/correlational results on observational data (200 podcast episodes): they show that host and guest gender are associated with different dialogue-act patterns, not that gender causes those patterns.
- Sample sizes are 50 episodes per dyad; some effects (e.g. `answer_asymmetry`, driven entirely by `FEMALE->FEMALE`) rest on a single dyad group and would benefit from a larger sample before treating the finding as robust.

10. `src/compute_bias_features_v2.py`

10.1 Purpose

The manual audit in Section 5 showed that the hand-built lexicons in `compute_bias_features.py` (Section 1) were unreliable: only 12 of 50 sampled "directive" matches were real directives. This script replaces the four hand-built lexicons with two published resources and adapts them to spontaneous podcast speech using repeated precision audits (Section 11). It produces one row of conversational style features per speaker per episode, over all 418 collected episodes rather than the balanced 200-episode sample.

The two resources are Hyland's (2005) metadiscourse lexicon of hedges and boosters, and the ConvoKit PolitenessStrategies classifier (Danescu-Niculescu-Mizil et al., 2013), which marks 21 politeness strategies (gratitude, apologizing, deference, factuality, direct questions, 1st/2nd person, and others) using a dependency parse of each turn.

10.2 Methodology

- Input: the 418 WhisperX transcripts with diarized speakers, speaker gender, and HOST/GUEST labels (`data/outputs/whisperx/*_whisperx_diarized.gender.host_guest.json`).
- Turns: consecutive segments from the same speaker are merged into one turn before scoring. Segments without a diarized speaker are dropped, and speech in the first and last 60 seconds of each episode (intro/outro monologues and sign-off thank-yous) is excluded.
- Hyland hedges and boosters (`data/lexicons/hyland_2005_hedges_boosters.json`; 101 hedges, 64 boosters): all terms are compiled into one case-insensitive, whole-word regular expression, longest term first. Each stretch of text is matched at most once, so "no doubt" counts as a booster and does not also count "doubt" as a hedge.

- Spoken-speech adjustments to Hyland, stored in the same file under `spoken_adjustments`: (1) discourse-marker phrases such as "you know", "I don't know", "talk about" and "think about" are matched and discarded, so they cannot count as hedges or boosters; (2) "I think" is counted as a hedge, following Prokofieva & Hirschberg (2014) on hedging in conversation; (3) terms that audited as mostly literal in speech are dropped, e.g. hedges about, around, would, could, should, feel, quite and boosters know, think, thought, find, show, always, never, must, true, believe (with their inflected forms).
- ConvoKit: each episode's turns are loaded into a ConvoKit Corpus, parsed with spaCy (`en_core_web_sm`), and scored with `PolitenessStrategies` at the marker level, so every occurrence is counted, not just whether a strategy appears in a turn.
- ConvoKit adjustments (`data/lexicons/convokit_adjustments.json`): Factuality no longer counts "really" or "the point"; Deference no longer counts "good"; and `Direct_question` only counts a wh-question when its sentence ends with "?" and is the last sentence of the turn, so that the other speaker responds next. The reasons and audit counts for each change are recorded in the JSON files and summarized in Section 11.
- For each speaker in each episode, the script writes the word count, turn count, and for every feature the raw count and the rate per 1,000 words. ConvoKit features also get a turn rate: the share of the speaker's turns with at least one occurrence.
- Alongside the features, it writes a long table of which terms or markers produced each count (for tracing any feature back to its words), and a random sample of 50 hits per feature with the surrounding turn text, used for the precision audits.

10.3 How It Was Run

The script runs on the cluster as a CPU-only SLURM job (2 cores, 8 GB, about 21 minutes for 418 episodes) in the convokit conda environment:

```
sbatch sbatch/run_bias_features_v2.sbatch
```

which runs:

```
python src/compute_bias_features_v2.py --input_glob
"data/outputs/whisperx/*_whisperx_diarized.gender.host_guest.json" --out_csv
results/features/speaker_features_v2.csv --terms_csv
results/features/speaker_feature_terms_v2.csv --audit_csv
results/features/audit_sample_v2.csv --audit_n 50
```

Both sets of adjustments are on by default; `--no_spoken_adjustments` and `--no_convokit_adjustments` run the lexicons as published.

10.4 Output

Output file: `results/features/speaker_features_v2.csv` — 836 rows × 73 columns (one HOST row and one GUEST row for each of the 418 episodes), with no missing values.

File	Rows	Contents
------	------	----------

speaker_features_v2.csv	836	Features per (episode, speaker)
speaker_feature_terms_v2.csv	155,750	Count of each term/marker per (episode, speaker, feature)
audit_sample_v2.csv	1150	50 random hits per feature, with turn text

Columns produced:

- episode_id, speaker_id, role, gender, word_count, turn_count.
- hy_hedge_count, hy_hedge_per_1k, hy_booster_count, hy_booster_per_1k (Hyland).
- For each of the 21 ConvoKit strategies: ck_<strategy>_count, ck_<strategy>_per_1k, ck_<strategy>_turn_rate (e.g. ck_factuality_per_1k, ck_direct_question_turn_rate).

10.5 Dataset Composition of the Output

Role	Female	Male	Unknown	Total
HOST	125	293	0	418
GUEST	128	288	2	418

Guests speak far more than hosts: the median guest speaks 8,833 words per episode, against 2,388 for the median host. The two guests with unknown gender are resolved by manual overrides in Section 12.

10.6 Descriptive Statistics

Rates per 1,000 words for the five features carried into the dyad analysis (Sections 13-15), by role, with the words that account for most of each feature's hits:

Feature	Host mean / median	Guest mean / median	Top terms (share of hits)
Hedges	5.55 / 5.00	6.51 / 5.36	i think 41%, maybe 11%, sometimes 8%
Boosters	4.05 / 3.73	4.24 / 3.56	really 32%, actually 22%, of course 10%
Factuality	1.32 / 1.15	1.21 / 0.91	actually 71%, in fact 14%, honestly 7%
Deference	0.26 / 0.00	0.06 / 0.00	interesting 39%, nice 31%, great 22%
Direct questions	4.61 / 3.92	0.29 / 0.17	what 56%, how 26%, why 14%

The spoken-speech exclusions matter: 30,742 phrase matches were discarded ("you know" alone is 61% of them), which is about as many as all counted hedges. Direct questions are almost entirely a host behavior once rhetorical questions are removed. Deference is rare: most speakers never use it, so its median is 0.

10.7 Notes & Limitations

- Word lists cannot read context. The largest remaining source of error is "actually" used as filler ("thank you for actually being here"); it is 71% of Factuality hits and 22% of booster hits. It was kept because it is correct in roughly three quarters of audited uses.
- ConvoKit's strategies were designed on written requests (Wikipedia and Stack Exchange); podcast speech has more fillers, rhetorical questions, and backchannels, which is why the adjustments above were needed.
- Deference in speech is mostly acknowledgement ("Interesting.", "Nice.") rather than explicit praise, so it is better described as acknowledgement.
- Direct_question requires the question to end the turn, which removes about half of the real questions along with the rhetorical ones (Section 11). Rates are comparable across groups but undercount questions in absolute terms. Guest direct questions are not reliable and are not used.
- Diarization errors that merge two speakers into one turn can attribute one person's words to the other.

11. Precision Audits of the v2 Features (src/make_audit_sample.py)

11.1 Purpose

Section 5 showed that a feature can look plausible and still be mostly wrong. Before using the v2 features, each one was checked by hand: a random sample of the words the script counted was read in context, and each hit was marked correct or not. The results drove the adjustments in Section 10.2, and the audit was repeated after each change.

11.2 Methodology

- compute_bias_features_v2.py samples 50 random hits per feature (reservoir sampling) into results/features/audit_sample_v2.csv.
- make_audit_sample.py selects the features being audited, rebuilds each hit's context from the full turn with the matched words in `[[ ]]`, and adds a yes/no question for each feature. For example, for hedges: "Does `[[ ]]` express uncertainty, tentativeness, approximation, or reduced commitment?"; for direct questions: "Is this a direct wh-question actually asked of someone?"
- Each hit is labeled 1 (correct) or 0, with a short note for every 0. Precision is the share of hits labeled 1. Rhetorical and self-answered questions, reported speech, literal uses ("make sure", "the point where"), and filler uses count as 0.
- The labels were made by an AI assistant (Claude) following the per-feature questions, and are marked as such in the labeled_by column of each audit file. They have not yet been checked by a human annotator (see 11.6).

11.3 How It Was Run

```
python src/make_audit_sample.py --features  
hy_hedge,hy_booster,ck_factuality,ck_direct_question,ck_deference --out  
results/audit/audit_v2e_to_label.csv
```

11.4 Audit Rounds

Each round sampled fresh hits from the pipeline as it stood at the time (50 per feature).

Precision per round:

Round	Pipeline	Hedge	Boost.	Fact.	Def.	Q	Change made after
1 (v2)	Published lists + discourse-marker exclusions, "I think" as hedge	0.46	0.34	0.44	0.66	0.44	Drop Hyland terms that were mostly literal (about, would, feel, know, always, thought, ...); drop Factuality "really", Deference "good"; questions must end with "?"
2 (v2b)	After round 1 changes	1.00	0.70	0.84	0.72	0.32	Drop booster "believe" (mostly religious or others' beliefs)
3 (v2c)	After round 2 changes	0.84	0.74	0.76	0.84	0.58	Questions must end the turn; drop Factuality "the point" (0 correct in every round)
4 (v2d)	Final pipeline	—	—	—	—	0.86	None (direct questions only)
5 (v2e)	Final pipeline, fresh sample	0.86	0.68	0.68	0.80	0.86	None

Hedge = hedges, Boost. = boosters, Fact. = factuality, Def. = deference, Q = direct questions.

Round 1 also covered gratitude (0.88) and apologizing (0.80), which were accurate and needed no change; they are not used in the dyad analysis. Round 4's direct-question hits are the same 50 as round 5's.

11.5 Final Precision

For hedges, boosters and deference, rounds 3 and 5 sample the same final lexicon, and factuality only lost "the point", so they are pooled (about 100 hits each; round 3's "the point" hits removed). Direct questions use round 4, the only sample after its last change.

Feature	Hits	Precision (95% CI)	Host	Guest
---------	------	--------------------	------	-------

Hedges	99	0.86 (0.78-0.91)	0.76 (n=21)	0.88 (n=78)
Boosters	100	0.71 (0.61-0.79)	0.56 (n=27)	0.77 (n=73)
Factuality	95	0.76 (0.66-0.83)	0.65 (n=20)	0.79 (n=75)
Deference	98	0.82 (0.73-0.88)	0.92 (n=53)	0.69 (n=45)
Direct questions	50	0.86 (0.74-0.93)	0.97 (n=40)	0.40 (n=10)

All five features are between 0.71 and 0.86 overall. Two findings shaped the dyad analysis: guest direct questions stayed unreliable (mostly rhetorical questions inside stories), so direct questions are analyzed for hosts only; and booster and deference precision differs between hosts and guests, so the analysis compares each role separately across dyads rather than subtracting guest rates from host rates (Section 13.2). In the audit samples, precision for female and male speakers was similar, but there were only 5-16 female speakers per feature.

11.6 Notes & Limitations

- With 50 hits, a single round's precision is uncertain by about ± 0.10 ; changes between rounds smaller than that are within noise.
- The rule that questions must end the turn was chosen using the round 3 labels, which overstates how well it works on those labels (0.88). Round 4 checks it on new hits (0.86).
- Labels were made by an AI assistant and one pass only; a human check of a subset (for example 50 hits per feature, with inter-annotator agreement) would make these numbers citable.
- The audit snippet marks the first place the term appears in the turn, which is not always the hit that was counted (e.g. a turn with several questions). Affected rows were labeled from the real hit and noted.

12. src/episode_dyads.py

12.1 Purpose

Assigns every episode its host→guest gender dyad, using all 418 episodes instead of the balanced 50-per-dyad sample from Section 2 (balanced_dataset.py stays available for robustness checks).

12.2 Methodology

- From speaker_features_v2.csv, take the gender of the HOST and of the GUEST speaker with the most words in each episode.
- Apply manual guest-gender corrections for the two episodes where gender detection returned "unknown" (ep_174 and ep_261, both male after review).
- Keep episodes whose dyad is one of MALE->MALE, MALE->FEMALE, FEMALE->MALE, FEMALE->FEMALE (host gender first); none had to be dropped.

12.3 How It Was Run

```
python src/episode_dyads.py --features results/features/speaker_features_v2.csv --out results/features/episode_dyads.csv
```

12.4 Output

Output file: results/features/episode_dyads.csv — 418 rows (episode_id, host_gender, guest_gender, dyad).

Dyad	Episodes
MALE->MALE	226
MALE->FEMALE	67
FEMALE->MALE	64
FEMALE->FEMALE	61
Total	418

12.5 Who the Hosts Are

The corpus has only 9 hosts, identified by matching the host's voice to the speaker library (data/speaker_library.json). Every host has one gender, so all female-host episodes come from 4 hosts and all male-host episodes from 5:

Host	MM	MF	FM	FF	Total
Faye Dsouza	0	0	19	14	33
Karishma Mehta	0	0	28	32	60
Pooja Dhingra	0	0	4	5	9
Soha Ali Khan	0	0	1	1	2
Nikhil Kamath	15	0	0	0	15
Prakhar Gupta	18	5	0	0	23
Raj Shamani	117	32	0	0	149
Ranveer A	49	26	0	0	75
Samdish Bhatia	19	4	0	0	23
Not matched	8	0	12	9	29

MM = MALE->MALE, MF = MALE->FEMALE, FM = FEMALE->MALE, FF = FEMALE->FEMALE. One host, Raj Shamani, hosts 149 episodes, including 117 of the 226 MALE->MALE ones. In 29 episodes the host was labeled by the fallback method rather than a library match, so their identity is not recorded.

12.6 Notes & Limitations

- The dyads are unbalanced (226 MALE->MALE against 61-67 in each other group). The tests in Sections 14-15 are rank-based and do not assume equal group sizes.

- Because host gender is tied to 9 individual hosts and their shows, any difference between male-host and female-host dyads may be about those particular hosts rather than gender. Section 15 adds a test that holds the host fixed.

13. src/dyad_analysis_v2.py

13.1 Purpose

Builds the episode-level dyad dataset from the v2 features (the v2 counterpart of Sections 3 and 4), with descriptive summaries by dyad and by host. The statistical tests on this dataset are separate scripts in src/stat_analysis/ (Sections 14 and 15).

13.2 Methodology

- Join speaker_features_v2.csv to episode_dyads.csv, and put each episode's host and guest values side by side (one row per episode).
- Metrics, all per 1,000 words: hedges, boosters, factuality and deference for both host and guest; direct questions for the host only (Section 11.5). Plus host speaking share (host words / host + guest words) and dominance ratio (host words / guest words).
- No host-minus-guest asymmetry columns. Because booster and deference precision differs between hosts and guests (Section 11.5), subtracting one from the other would mix two different error rates. Each role is instead compared across dyads, e.g. host hedging with male guests vs. female guests.
- Host identity (host_name) is read from the speaker-library match in each transcript.

13.3 How It Was Run

```
python src/dyad_analysis_v2.py
```

All paths default to the files above.

13.4 Output Files

File	Contents
results/dyads/dyad_analysis_v2.csv	418 rows: dyad, host name, host/guest metrics
results/dyads/dyad_median_mad_stats_v2.csv	Median, MAD, n per dyad per metric
results/dyads/host_level_medians_v2.csv	Median of each metric for each of the 9 hosts

13.5 Medians by Dyad

Metric	MM	MF	FM	FF
Host speaking share	0.22	0.28	0.18	0.20
Host hedges	5.04	5.46	5.04	4.64
Guest hedges	4.37	4.86	8.21	7.13
Host boosters	3.05	3.51	5.28	5.98

Guest boosters	2.47	3.46	4.70	6.11
Host factuality	0.96	1.19	1.55	1.28
Guest factuality	0.74	0.96	1.03	1.75
Host deference	0.00	0.24	0.00	0.00
Guest deference	0.00	0.00	0.00	0.00
Host direct questions	3.33	5.15	6.36	3.70

Deference medians are 0 in every dyad because most speakers never use it; the share of episodes where the host uses it at least once is MM 46%, MF 60%, FM 5%, FF 10%.

13.6 Medians by Host

Host	Sex	Eps.	Host share	Host hedges	Host boosters	Host fact.	Host dir. Q	Guest boosters
Faye Dsouza	F	33	0.19	4.12	5.85	1.83	4.15	4.59
Karishma Mehta	F	60	0.18	4.67	5.82	1.09	6.62	5.13
Pooja Dhingra	F	9	0.24	7.53	6.24	2.07	3.43	7.51
Soha Ali Khan	F	2	0.22	8.79	4.24	0.86	2.36	3.42
Nikhil Kamath	M	15	0.22	10.93	3.59	1.03	3.45	6.43
Prakhar Gupta	M	23	0.30	5.31	3.79	1.52	2.38	2.76
Raj Shamani	M	149	0.22	4.14	2.28	0.85	3.88	2.23
Ranveer A	M	75	0.25	7.81	4.04	1.43	3.79	3.17
Samdish Bhatia	M	23	0.22	4.77	3.26	0.47	4.96	2.79

Hosts of the same gender differ as much as the two genders do. For example, host hedging ranges from 4.1 to 10.9 per 1,000 words among male hosts, and from 4.1 to 8.8 among female hosts. This is why the host-gender comparisons in Sections 14-15 need to be read as comparisons between these particular hosts.

14. [src/stat_analysis/kruskal_wallis_test_v2.py](#)

14.1 Purpose

Tests whether each metric in results/dyads/dyad_analysis_v2.csv (Section 13) differs across the four gender dyads at all, before asking which dyads differ (Section 15). This is the v2 counterpart of Section 8.

14.2 Methodology

- For each of the 11 metrics in Section 13.2, run scipy's `kruskal()` across the four dyad groups (non-parametric, rank-based, no normality assumption).

- Correct the 11 p-values for multiple testing with the Benjamini-Hochberg false discovery rate (FDR), and call a metric significant if its corrected p-value is below 0.05. (Section 8 did not correct across metrics.)
- Save H, raw p, corrected p, and the median of each dyad to results/stat_analysis/kruskal_wallis_v2_results.csv.

14.3 How It Was Run

python src/stat_analysis/kruskal_wallis_test_v2.py

Paths default to the files above. The Benjamini-Hochberg correction is in src/stat_analysis/fdr.py, shared with the Section 15 scripts (statsmodels is not installed in the convokit environment).

14.4 Results

Metric	H statistic	p-value	FDR p	Significant
Host speaking share	50.50	<0.0001	<0.0001	Yes
Dominance ratio	50.50	<0.0001	<0.0001	Yes
Host hedges	3.26	0.3528	0.3528	No
Guest hedges	50.44	<0.0001	<0.0001	Yes
Host boosters	96.56	<0.0001	<0.0001	Yes
Guest boosters	74.27	<0.0001	<0.0001	Yes
Host factuality	12.68	0.0054	0.0059	Yes
Guest factuality	50.76	<0.0001	<0.0001	Yes
Host deference	65.54	<0.0001	<0.0001	Yes
Guest deference	20.25	0.0002	0.0002	Yes
Host direct questions	34.74	<0.0001	<0.0001	Yes

10 of the 11 metrics differ significantly across dyads. Only host hedging does not ($p = 0.35$): hosts hedge at similar rates in all four dyads.

14.5 Notes & Limitations

- Host speaking share and dominance ratio are two transformations of the same word counts, so their rank-based results are identical; they are one finding, not two.
- A significant result means at least one dyad differs. Because host gender is tied to 9 specific hosts (Section 12.5), a significant host-side metric can reflect differences between those hosts rather than gender. Section 15 separates the two for guest gender.

15. src/stat_analysis/Mann_Whitney_test_v2.py and within_host_test_v2.py

15.1 Purpose

Identifies which pairs of dyads differ for each significant metric (the v2 counterpart of Section 9), and then checks which differences survive when the host is held fixed. Since each host interviews both female and male guests, guest-gender effects can be tested within each host's show; host-gender effects cannot.

15.2 Methodology

- Pairwise (Mann_Whitney_test_v2.py): for each of the 10 metrics significant in Section 14 (read from `kruskal_wallis_v2_results.csv`), run a two-sided Mann-Whitney U test on all 6 dyad pairs, correct the 6 p-values with Benjamini-Hochberg FDR within each metric (as in Section 9), and compute the rank-biserial correlation as effect size. Note the sign: here a positive value means dyad A is higher, the opposite of the convention in Section 9.
- Within host (within_host_test_v2.py): episodes are ranked within each host (as a percentile, 0-1) on each metric. For each host, compute the mean rank of female-guest episodes minus the mean rank of male-guest episodes, then average across hosts, weighting hosts with more of both guest genders more heavily. A value of +0.15 means female-guest episodes sit, on average, 15 percentile points higher within their host's episodes.
- The p-value comes from a permutation test: guest gender is shuffled within each host 10,000 times, and p is the share of shuffles with an effect at least as large. P-values are FDR-corrected across the 10 metrics. Episodes without a library-matched host are left out (8 hosts with both guest genders, 374 episodes).
- Exploratory: the same within-host effect is computed separately for male and female hosts, and their difference is tested with the same shuffles (does the guest-gender effect depend on the host's gender?).

15.3 How It Was Run

```
python src/stat_analysis/Mann_Whitney_test_v2.py
```

```
python src/stat_analysis/within_host_test_v2.py
```

Mann_Whitney_test_v2.py must run after `kruskal_wallis_test_v2.py`, since it tests only the metrics that were significant there. `within_host_test_v2.py` takes `--n_perm` (default 10,000) and `--seed` (default 42) for the permutation test. Outputs: `results/stat_analysis/pairwise_dyad_tests_v2.csv` and `results/stat_analysis/within_host_guest_gender_v2.csv`.

15.4 Results: Pairwise Differences Between Dyads

Of the 54 pairwise tests (9 metrics × 6 pairs, leaving out dominance ratio, which repeats speaking share), 38 are significant after FDR correction. Each cell shows the rank-biserial effect size for dyad A vs. dyad B (positive = A higher); * marks a significant pair.

Metric	MM vs MF	MM vs FM	MM vs FF	MF vs FM	MF vs FF	FM vs FF
Host speaking share	-0.37*	+0.32*	+0.18*	+0.65*	+0.54*	-0.16
Guest hedges	-0.15	-0.48*	-0.40*	-0.42*	-0.32*	+0.07
Host boosters	-0.17*	-0.50*	-0.72*	-0.39*	-0.62*	-0.19
Guest boosters	-0.23*	-0.44*	-0.62*	-0.28*	-0.49*	-0.23*
Host factuality	-0.07	-0.24*	-0.21*	-0.18	-0.14	+0.04
Guest factuality	-0.18*	-0.28*	-0.57*	-0.13	-0.45*	-0.23*
Host deference	-0.14	+0.40*	+0.38*	+0.53*	+0.51*	-0.05
Guest deference	-0.01	+0.25*	+0.22*	+0.26*	+0.24*	-0.03
Host direct questions	-0.27*	-0.44*	-0.12	-0.21	+0.16	+0.33*

The biggest differences separate male-host from female-host dyads (MM and MF vs. FM and FF): boosters are higher in female-host episodes for both hosts and guests, factuality is higher in female-host episodes than in MALE->MALE ones, and host deference is higher in male-host episodes. Host speaking share is highest in MF episodes (median 0.28, vs. 0.18-0.22 elsewhere). Host direct questions are highest in FM episodes.

15.5 Results: Guest Gender Within Host

Metric	Effect (female – male guests)	p (perm.)	FDR p	Significant	Hosts in same direction
Host speaking share	+0.149	<0.0001	0.0003	Yes	6 of 7
Host hedges	+0.013	0.6898	0.8623	No	5 of 7
Guest hedges	+0.055	0.1010	0.1683	No	5 of 7
Host boosters	+0.085	0.0118	0.0236	Yes	6 of 7
Guest boosters	+0.154	<0.0001	0.0003	Yes	7 of 7
Host factuality	-0.001	0.9871	0.9871	No	5 of 7
Guest factuality	+0.131	<0.0001	0.0003	Yes	6 of 7
Host deference	+0.067	0.0111	0.0236	Yes	5 of 7
Guest deference	+0.004	0.9044	0.9871	No	3 of 7
Host direct questions	+0.025	0.4584	0.6548	No	3 of 7

"Hosts in same direction" counts the hosts whose female-guest episodes rank higher, out of the 7 hosts with at least 4 episodes of each guest gender (Soha Ali Khan, with one of each, is left out of the count but not the test).

15.6 Results: Does the Guest-Gender Effect Depend on Host Gender? (Exploratory)

Metric	Effect, male hosts	Effect, female hosts	p (perm.)	FDR p
Host speaking share	+0.169	+0.112	0.4196	0.9332

Host hedges	+0.028	-0.015	0.5464	0.9332
Guest hedges	+0.106	-0.043	0.0359	0.1795
Host boosters	+0.093	+0.070	0.7499	0.9332
Guest boosters	+0.151	+0.157	0.9332	0.9332
Host factuality	+0.002	-0.005	0.9299	0.9332
Guest factuality	+0.120	+0.152	0.6605	0.9332
Host deference	+0.092	+0.021	0.1283	0.4276
Guest deference	-0.003	+0.016	0.7240	0.9332
Host direct questions	+0.145	-0.205	<0.0001	0.0010

Only host direct questions differ after correction: male hosts ask female guests more direct questions than male guests, while female hosts ask female guests fewer. This test was added after the per-host results showed the pattern, so it should be confirmed before being reported as a finding.

15.7 Interpretation: What These Results Suggest

- **Hosts take more of the floor with female guests.** Holds within host: hosts talk more when the guest is a woman (+0.15, in 6 of 7 hosts). The pooled result (MF highest) is therefore not just a difference between shows.
- **Female guests speak with more emphasis and certainty, not more hedging.** Within the same show, female guests use more boosters (+0.15, all 7 hosts) and more factuality markers (+0.13) than male guests. They do not hedge more: the pooled difference in guest hedging (higher in female-host dyads) disappears within host, so it came from the shows, not the guests' gender.
- **Hosts adjust slightly to female guests.** Within host, hosts use more boosters (+0.09) and more acknowledgements (+0.07) with female guests. Host hedging and host factuality do not change with guest gender.
- **Host-gender differences cannot be separated from who the host is.** Female-host episodes have more boosters and factuality, and male-host episodes more deference, but these are differences between 4 and 5 individual hosts, and hosts of the same gender vary as much as the genders do (Section 13.6). They cannot be attributed to host gender with this corpus.
- **Direct questions may follow a cross-gender pattern.** Pooled, FM episodes have the most host direct questions, which agrees with Section 9's finding that female hosts ask more (host_ask), but FF episodes do not. The exploratory within-host result suggests hosts ask more direct questions of guests of the other gender. This needs confirming.

15.8 Notes & Limitations

- The within-host test answers a narrower question than the pooled test: whether the same host behaves differently with female and male guests. It cannot test host gender, because no host appears in both host-gender groups.

- These are associations in observational data. Guests are not randomly assigned to hosts, so female and male guests on the same show may also differ in profession or topic (compare the topic-control analyses), which could explain part of the guest-gender effects.
- Feature precision is 0.71-0.86 (Section 11). Errors that are the same across dyads weaken differences rather than create them; errors that differ by guest gender could not be ruled out with the small number of female speakers in the audits.
- FDR correction is applied within each test family (11 omnibus metrics; 6 pairs per metric; 10 within-host metrics), not across all tests together.